

Variables y Tipos de Datos

En el mundo del desarrollo, nuestra materia prima es la información. Para poder manipularla, transformarla y mostrarla, necesitamos un lugar donde guardarla de forma segura y organizada. En esta lección, vamos a sumergirnos en el concepto de **variables** entendiendo que son el pilar fundamental de cualquier algoritmo que construyamos.

Variables

Las **variables** son espacios de memoria que reservamos para almacenar **datos**. Podemos imaginarlas como "*cajas*" *etiquetadas* dentro de nuestro código donde guardamos valores como *números enteros, números con decimales, palabras, listas*, entre otros.

Por ejemplo, si escribimos:

```
1 nombre = "Lyfter"
```

[Copiar](#)

Estamos diciendo: Python, toma una **caja** con la etiqueta **nombre** y guarda dentro el texto **Lyfter**. Más adelante, cuando necesites esa información, Python abrirá la caja correcta.

Sin ellas, nuestro código sería muy repetitivo, complejo y nuestra computadora no podría recordar mucha de la información que procesa.

Identificadores

Para crear una de estas cajas, necesitamos *etiquetarla* con un nombre **único** llamado **identificador**. Usamos el símbolo **=** para realizar una **asignación**, indicando que el **valor** de la derecha ahora se puede identificar con el **identificador** de la izquierda.

En Python, definir una **variable** es tan sencillo como:

```
identificador = valor de la variable
```

Acá algunos ejemplos:

```
1 # el identificador es my_variable
2 # su valor es 7
3 my_variable = 7
4
5 # el identificador es my_second_variable
6 # su valor es 26
7 my_second_variable = 26
8
9 print(my_variable)
```

Copiar

✓ 7

Las **variables** también pueden utilizarse para guardar **resultados** de otras operaciones, como sumas o restas. Por ejemplo:

```
1 # el identificador es my_variable
2 # su valor es 7
3 my_variable = 7
4
5 # el identificador es my_second_variable
6 # su valor es 26
7 my_second_variable = 26
8
9 # el identificador es result
10 # su valor es 7 + 26
11 result = my_variable + my_second_variable
12
13 print(result)
```

Copiar

✓ 33

Más adelante vamos a ver qué otras operaciones se pueden hacer con los distintos **tipos de datos** que existen, así como los estándares para definir los **identificadores** en Python.

Sintaxis

Al igual que en los lenguajes humanos, que tienen sus normas de gramática y ortografía para velar por su comprensión y buen uso, los lenguajes de programación tienen sus propias reglas que debemos seguir. Sin estas reglas no se podría ejecutar nuestro programa. A este conjunto de reglas se le llama la **sintaxis** y es distinta para cada lenguaje.



Por ejemplo, en Python existe la sintaxis *hashtag* o *numeral* `#` que nos permite agregar **comentarios** al código. Es decir, anotaciones que no son consideradas como parte del código. Todo lo que se escriba después de un `#` es ignorado por Python.

Para que la **sintaxis** de nuestros **identificadores** en Python sea correcta, debemos seguir las siguientes normas:

No pueden iniciar con números: Podemos usar números, pero nunca al inicio del nombre.

```
1 6number = 6 # Error porque no deben iniciar con números. Copiar
```

✗ SyntaxError: invalid decimal literal

No pueden contener símbolos ni espacios: No usamos `@`, `$`, `%` ni espacios en blanco. El **único** carácter especial permitido es el guion bajo (`_`).

```
1 apples&cookies = 46 # Error porque no deben contener símbolos Copiar
```

✗ SyntaxError: cannot assign to expression here. Maybe you meant '==' instead of '='?

```
1 my variable = 4 # Error porque no deben tener espacios. Copiar
```

✗ SyntaxError: invalid syntax

Sensibilidad de mayúsculas (Case Sensitive): Para Python, `MiCaja` y `micaja` son dos variables totalmente distintas.

```
1 myvariable = 402
2 myVariable = 56
3 MyVariable = 103
4
5 print(myVariable)
```

Copiar

✓ 56

Tipos de Datos 🎨

El siguiente paso es entender qué **tipo de información** podemos guardar en las **variables**. Cada **dato** pertenece a una categoría que define qué operaciones podemos realizar con él. Todos los lenguajes de programación tienen **tipos de datos** distintos, aunque muy parecidos.



Aprender el nombre de cada **tipo de dato** es **indispensable** para cualquier desarrollador. No podemos hablar de "números enteros". Debemos hablar de "ints".

Números Enteros (**int**)

Los **integers**, **int** son números **enteros sin decimales**, ya sean positivos o negativos. Los utilizamos para representar cantidades exactas o contables.

Ejemplos de uso:

Edad de un usuario.

Cantidad de productos en un carrito.

Contador de un ciclo.

Ejemplo:

```
1 # Representamos datos exactos
2 special_number = 25
3 secret_number = -3
4 result = special_number + secret_number
5
6 print(result)
```

Copiar



22

Números con Decimales (**float**)

Los **floats** son números con decimales. Estos se definen con un número seguido de un punto `.` y los usamos cuando necesitamos precisión en medidas o cálculos financieros.

Ejemplos de uso:

Saldo de una cuenta bancaria (*dinero*).

Temperatura corporal.

Promedio de una calificación.

Ejemplo:

```
1 pi_value = 3.14
2 gravity = 9.8
3 result = pi_value + gravity
4
5 print(result)
```

[Copiar](#)

12.9400000000000001

Como vemos arriba, los **float** **no dan resultados exactos**. En vez de `12.94` dió `12.9400000000000001`.

Esto sucede en **todos** los lenguajes de programación debido a que los CPUs no son capaces de entender números decimales naturalmente (como vimos en la lección de **Computadoras**), y su representación en binario no es totalmente precisa.

Esto significa que tampoco podemos definir cuántos decimales tendrá un float. Para esto, se deben convertir primero a **cadenas de texto**.



Otra consideración importante es que los floats se definen utilizando un punto `.` para los decimales. Una coma `,` **no creará un **float**** sino una **tupla** (otro tipo de dato que veremos más adelante).

```
1 wrong_float = 3,14
2
3 print(wrong_float)
```

Copiar

✓ (3, 14)

Cadenas de Texto (**str**)

Los **strings** son secuencias de caracteres (*letras, números o símbolos*) envueltas en comillas `"` o `'`. Los usamos para cualquier tipo de información textual. Al definirlos es **súper importante** que recordemos usar las comillas. De lo contrario, Python pensará que estamos escribiendo el **identificador** de una **variable**.

Ejemplos de uso:

Correo electrónico.

Contraseñas (aunque se guarden cifradas, nacen como texto).

Nombres de productos.

Ejemplo:

```
1 # Podemos usar comillas dobles
2 email = "contacto@lyfter.com"
3
4 # Comillas sencillas
5 username = 'Admin123_!$'
6
7 # También tres comillas dobles para strings de múltiples líneas
8 favorite_poem = """
9     Las rosas son rojas,
10    los strings son texto,
11    y estas son múltiples líneas.
12 """
13
14 print(email)
15 print("string que imprimos directamente")
16 print(favorite_poem)
```

Copiar

✓ contacto@lyfter.com
string que imprimos directamente
Las rosas son rojas,
los strings son texto,
y estas son múltiples líneas.

Cualquier dato que esté dentro de comillas será un `string` por más que luzca similar a otro `tipo de dato`. Por ejemplo, esta suma es posible al ser dos `ints`:

```
1 my_number = 4
2 print(my_number + 6)
```

Copiar

✓ 10

Sin embargo, acá no es posible ya que estamos intentando sumar un `int` a un `string`:

```
1 my_not_number = "4"  
2 print(my_not_number + 6)
```

Copiar

✗ TypeError: can only concatenate str (not "int") to str

Más adelante veremos cómo manejar estos casos para poder hacer la suma correctamente...

Por otro lado, acá algunos ejemplos de **strings** inválidos que harán que nuestro código falle por errores de sintaxis:

```
1 # No envolvemos en comillas el texto  
2 incorrect_string = esto no es un string
```

Copiar

✗ SyntaxError: invalid syntax

```
1 # Tampoco envolvemos en comillas el texto  
2 print(esto tampoco es un string)
```

Copiar

✗ SyntaxError: invalid syntax

```
1 # Usamos comillas diferentes  
2 print('esto tampoco porque uso comillas distintas')  
3 print("esto tampoco porque uso comillas distintas')
```

Copiar

✗ SyntaxError: unterminated string literal (detected at line 1)

Verdadero o Falso (**bool**)

Los **booleanos** representan los dos estados de la lógica:

True (Verdadero)

`False` (*Falso*).

Son lo más cercano a los números binarios del código máquina (1 y 0), y los empleamos para controlar el flujo de nuestras aplicaciones mediante condiciones lógicas.

Ejemplos de uso:

Verificar si un usuario está logueado.

Verificar si un producto tiene stock.

Verificar si el modo oscuro está activado.

Ejemplo:

```
1 is_user_active = True
2 admin_permissions = False
3 dark_mode_on = True
```

Copiar

Listas (`list`)

Un `list` permite guardar múltiples datos o **elementos** ordenados en **una** sola estructura o variable. Además, en ellas podemos *agregar más elementos*, *quitar elementos*, o *cambiar elementos existentes*. Esto hace que sean **súnamente flexibles** y utilizadas en múltiples escenarios.

Ejemplos de uso:

Lista de notas de un estudiante.

Nombres de los meses.

Una lista de tareas pendientes.

Para crear una lista usamos paréntesis cuadrados `[]` con cada uno de sus **datos** dentro, separados por coma `,`.

Ejemplo:

```
1 grades_list = [85, 90, 78, 92]
2
3 print(grades_list)
```

Copiar

✓ `[85, 90, 78, 92]`

Cada uno de los elementos de una lista tiene un **índice** específico que hace referencia a su **posición** en la lista. Los índices inician desde **0** (no desde **1**), por lo que el primer elemento de la lista tendrá el índice **0**, el segundo el índice **1**, el tercero el índice **2**, y así sucesivamente.

Para acceder a un elemento específico de una lista, debemos usar paréntesis cuadrados con su **índice** adentro.

```
1 grades_list = [85, 90, 78, 92]
2 # Si quiero imprimir el primer elemento (85) uso el índice 0
3 print(grades_list[0])
4
5 # Entonces si 85 tiene el índice 0
6 # 90 tiene el índice 1
7 # 78 tiene el índice 2
8 # 92 tiene el índice 3
```

Copiar

✓ 85

Acá otro ejemplo de esto usando múltiples **listas**:

```
1 list_of_integers = [4, 7, 3, 9, 3]
2 list_of_strings = ["Hello", "I'm", "A", "String", "List"]
3 list_of_booleans = [True, True, False]
4
5 print(list_of_integers[3])
6 print(list_of_strings[1])
7 print(list_of_booleans[2])
```

Copiar

✓ 9
I'm
False

Tuplas (**tuple**)

Las **tuplas** son muy similares a las **listas**, con una diferencia clave: son **inmutables**. Esto significa que una vez que nosotros las creamos, no podemos cambiar sus elementos. Se definen con paréntesis regulares **()**.

Ejemplos de uso:

Coordenadas GPS (*latitud y longitud*).

Configuraciones que deben permanecer fijas.

Ejemplo:

```
1 awesome_tuple = (50.4, 35.3, 20.2, 5.1, -10)
2 print(awesome_tuple[2])
3
4 # Entonces si 50.4 tiene el índice 0
5 # 35.3 tiene el índice 1
6 # 20.2 tiene el índice 2
7 # 5.1 tiene el índice 3
8 # -10 tiene el índice 4
```

[Copiar](#)

✓ 20.2

Diccionarios (**dict**)

Los **diccionarios** también permiten guardar varios **valores** en una sola estructura, justo como las **listas** o **tuplas**. La diferencia **clave** es que los **valores** de los diccionarios no están en *orden* ni tienen **índices**, sino que tienen un **nombre** - como si se tratara de una variable con variables más pequeñas dentro.

A este nombre le llamamos llave o **key** y al **valor** le llamamos **value**. Juntos, reciben el nombre de **key : value pairs** o parejas de llave y valor. Para definir los **keys** se pueden usar datos de tipo **int** **bool** o incluso **tuplas**, pero lo más común es usar **strings**. Por otro lado, los **values** pueden ser cualquier otro **tipo de dato**, justo como en una **lista** o **tupla**.

Mientras que **listas** o **tuplas** se utilizan para guardar varios datos de una sola categoría (como números telefónicos o nombres), los **diccionarios** son más aptos para guardar distintos **datos** de distintos tipos relacionados entre sí.

Ejemplos de uso:

Información completa de un usuario.

Detalles de un vehículo.

Configuración de una aplicación.

Los diccionarios se definen utilizando llaves `{}` con sus `key : value pairs` dentro separados por comas `,`.

Ejemplo:

```
1 # Representación de un usuario
2 user_profile = {
3     "name": "John Doe",
4     "email": "john.doe@lyfter.com",
5     "age": 26,
6     "is_subscribed": True
7 }
8
9 # Accedemos al correo usando la key de "email"
10 print(user_profile["email"])
```

[Copiar](#)

✓ john.doe@lyfter.com

También podemos combinar los `diccionarios` y las `listas` para crear estructuras de datos mas complejas. Por ejemplo, aquí tenemos una lista de casas de un condominio. Cada casa es un `diccionario` y tiene distintas `key`, incluyendo la de `rooms` que tiene una `lista` de `strings`.

Copiar

```

1  condo_houses = [
2      {
3          "number": 93,
4          "area_m2": 125,
5          "rooms": [
6              "living_room",
7              "kitchen",
8              "main_sleeping_room",
9              "second_sleeping_room",
10             "bathroom",
11         ],
12     },
13     {
14         "number": 95,
15         "area_m2": 125,
16         "rooms": [
17             "living_room",
18             "kitchen",
19             "main_sleeping_room",
20             "second_sleeping_room",
21             "third_sleeping_room",
22             "bathroom",
23         ],
24     },
25 ]
26
27 print(condo_houses[1]['rooms'][4])

```

✓ third_sleeping_room

Resumen de tipos de datos 🍷

Nombre completo	Nombre corto	Descripción	Ejemplo
integer	<code>int</code>	Número entero	<code>my_int = 45</code>

Nombre completo	Nombre corto	Descripción	Ejemplo
float	<code>float</code>	Número con decimal	<code>my_float = 5.63</code>
string	<code>str</code>	Cadena de texto	<code>my_string = "Hola"</code>
boolean	<code>bool</code>	Verdadero o Falso	<code>my_bool = True</code>
list	<code>list</code>	Guarda múltiples datos ordenados por índices. Es modificable.	<code>my_list = [6, "John"]</code>
tuple	<code>tuple</code>	Guarda múltiples datos ordenados por índices. No es modificable.	<code>my_tuple = (6, "John")</code>
dictionary	<code>dict</code>	Guarda múltiples datos con sus propios keys (<i>llaves</i>).	<code>my_dict = { "key_1": "Hello", "key_2": "World", }</code>